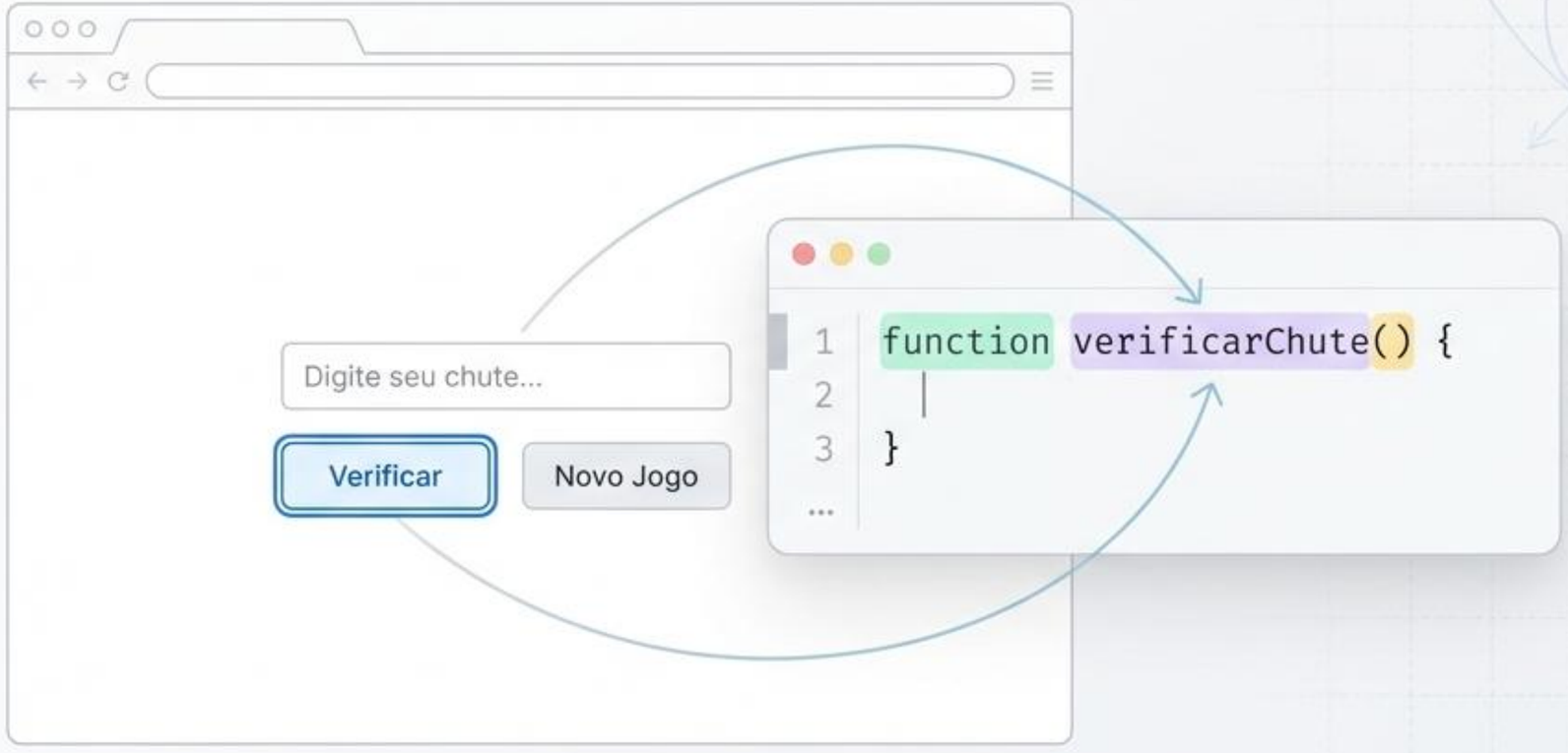
Desenvolvimento Front-End II: Lógica com JavaScript

Guia Prático: Estruturando o Jogo do Número Secreto



Do HTML estático à interatividade dinâmica.

NotebookLM

Como o JavaScript Encontra e Modifica o HTML

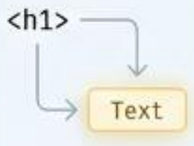
O escopo: representa a página inteira.

A ferramenta de busca: localiza o primeiro elemento que corresponda ao seletor.

```
document.querySelector('tag').innerHTML = 'texto';
```

```
<h1>Titulo</h1>
<p>Parágrafo</p>
<button>Clique</button>
```

O alvo: a propriedade HTML (ex: h1, p, button). Identifica o que será alterado (Case Sensitive).



A ação: acessa e substitui o conteúdo interno do elemento HTML selecionado.

NotebookLM

Injetando Textos Iniciais na Interface do Jogo

Wireframe Mapping

Jogo do número secreto

Escolha um número entre 1 e 10

Project Code

```
1
2  let titulo = document.querySelector('h1');
3  titulo.innerHTML = 'Jogo do número secreto';
4
5  let paragrafo = document.querySelector('p');
6  paragrafo.innerHTML = 'Escolha um número entre 1 e 10';
7
```

Key Takeaway: O uso de variáveis (**let**) armazena a seleção do DOM, permitindo a injeção de strings de texto diretamente na interface sem alterar o arquivo .html.

NotebookLM

A Anatomia de uma Função e seu **Gatilho** (Eventos)

O atributo HTML escuta o clique do usuário.

```
<button onclick="nomeDaFuncao()">
```

Chama a execução

Declaração

```
function nomeDaFuncao() {
  // Escopo: Responsabilidade única da função
}
```

Corpo da função

NotebookLM

Conectando o Botão Chutar à Lógica do Projeto

```
<button onclick="verificarChute()" class="container__botao">Chutar</button>
```

Chutar

JavaScript Function

```
1  function verificarChute() {
2      console.log('O botão foi clicado!');
3  }
```

```
> O botão foi clicado!
> O botão foi clicado!
> O botão foi clicado!
```

A função `verificarChute()` possui uma única responsabilidade inicial: capturar o evento de clique e confirmar a comunicação via console (ferramenta de diagnóstico do desenvolvedor).

NotebookLM

Evitando Repetição de Código com Parâmetros

O Problema - **Código Estático**

```
let titulo = document.querySelector('h1');
titulo.innerHTML = 'Jogo do número secreto';
titulo.innerHTML = 'Jogo do número secreto';

let paragrafo = document.querySelector('p');
paragrafo.innerHTML = 'Escolha um número entre
1 e 10';
```

A Solução - **Código Dinâmico**

```
function exibirTextoNaTela(tag, texto) {
    let campo = document.querySelector(tag);
    campo.innerHTML = texto;
}
```

Parâmetros (`tag`, `texto`) atuam como variáveis temporárias que recebem valores diferentes a cada chamada da função.

NotebookLM

Refatorando a Exibição de Textos no Jogo

```
1 exibirTextoNaTela('h1', 'Jogo do número secreto');
2 exibirTextoNaTela('p', 'Escolha um número entre 1 e 10');
```

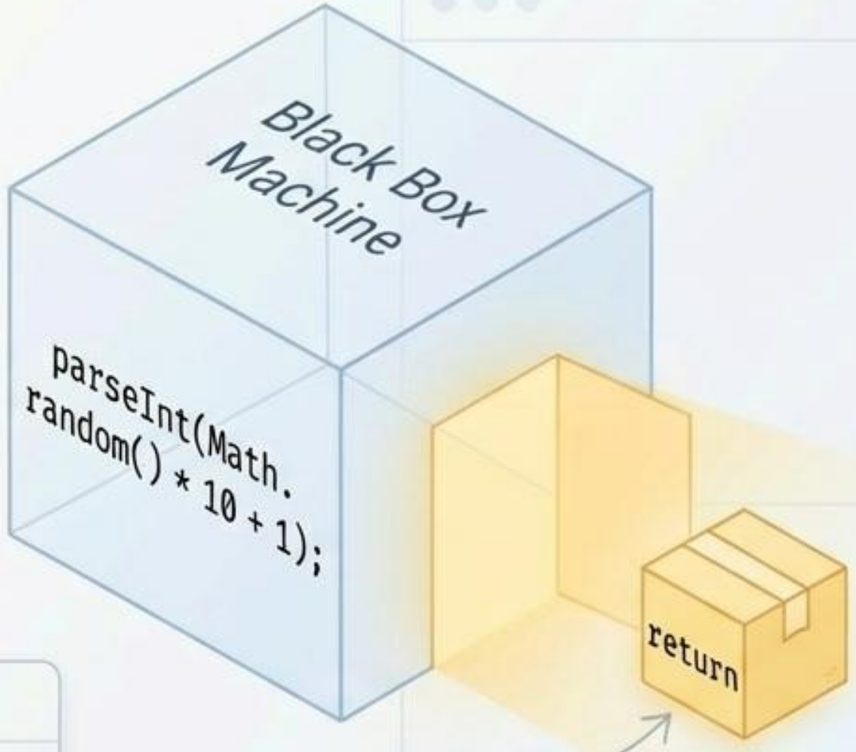
Jogo do número secreto

Escolha um número entre 1 e 10

Ao isolar o comportamento de exibição em uma única função, o escopo global fica limpo, legível e altamente escalável para futuras mensagens.

NotebookLM

Entregando Resultados com a Instrução "return"



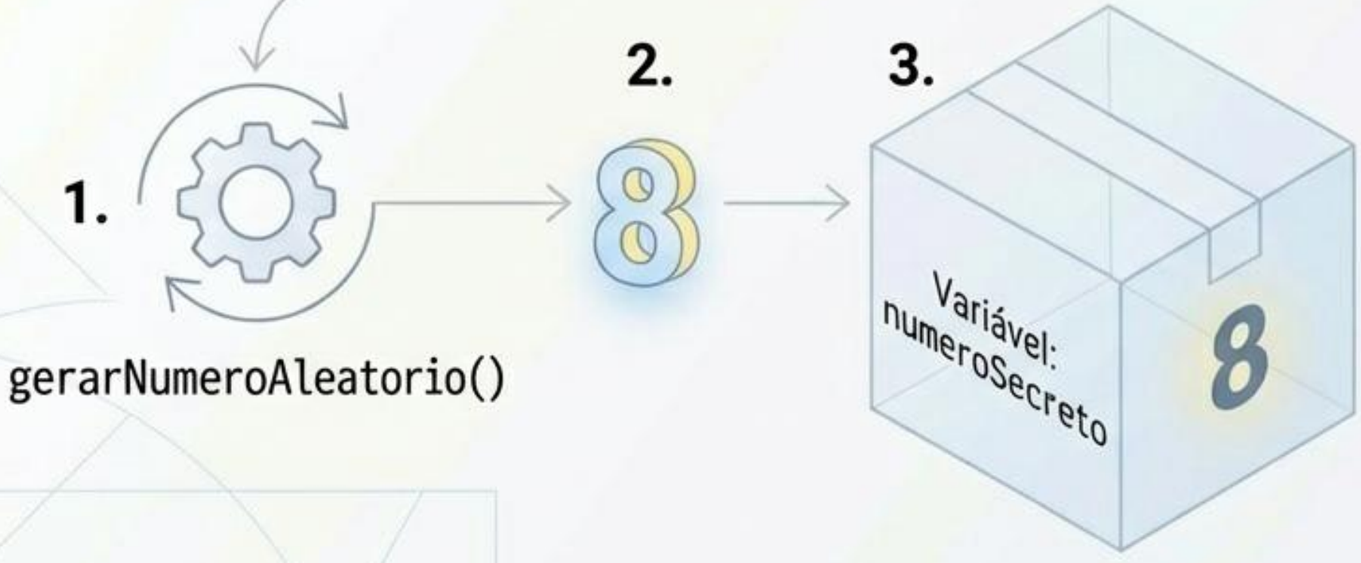

```
1  function gerarNumeroAleatorio() {
2      return parseInt(Math.random() * 10 + 1);
3  }
```

A palavra-chave `return` encerra a execução da função e devolve o valor processado para quem a invocou.

NotebookLM

Gerando e Armazenando o Número Secreto

```
1  let numeroSecreto = gerarNumeroAleatorio();
```



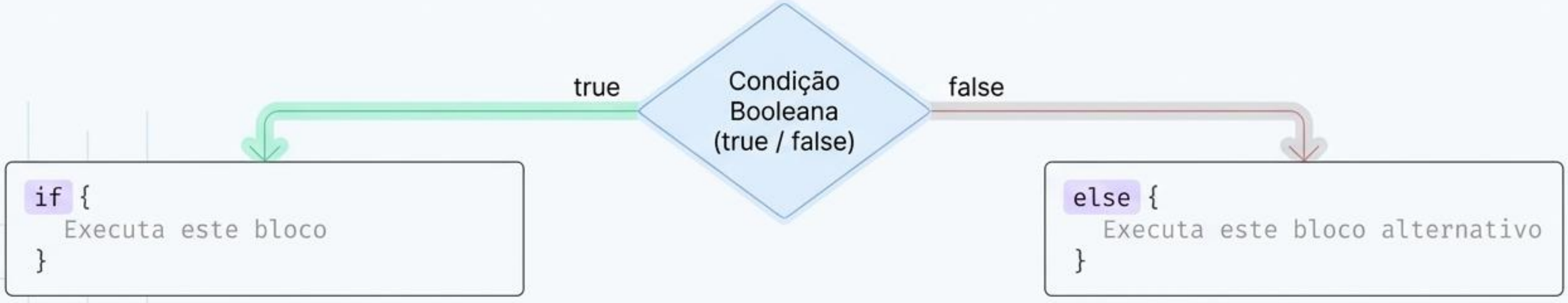
1. O código chama a função **gerarNumeroAleatorio()**.
2. A função calcula o número (ex: 8) e utiliza o **return**.
3. O valor retornado é capturado e permanentemente armazenado na variável global **numeroSecreto**.

```
> 8
```

NotebookLM

Operadores Relacionais e Lógica Condicional

	Operador	Nome	Função
1	`=`	Atribuição	Guarda um valor na memória.
2	`==`	Comparação	Verifica igualdade entre dois valores.



Booleanos: O núcleo lógico da programação, retornando exclusivamente verdadeiro (true) ou falso (false).

NotebookLM

Avaliando o Chute do Usuário

```
let chute = document.querySelector('input').value;
```

Extrai apenas o dado numérico digitado, ignorando as tags HTML da estrutura.

```
if (chute == numeroSecreto) {
  exibirTextoNaTela('h1', 'Acertou!');
} else {
  // Lógica de erro (maior ou menor)
}
```

NotebookLM

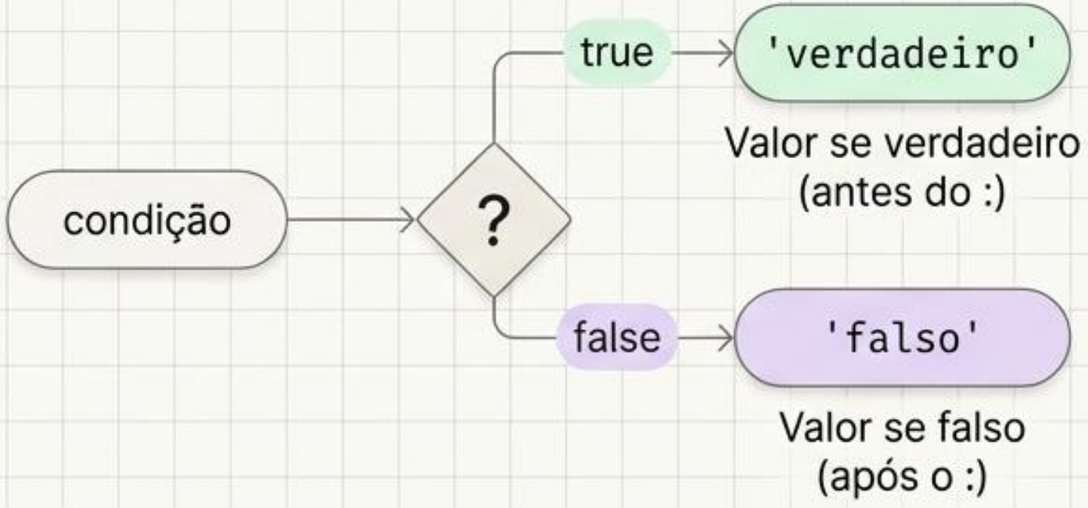
Template Strings e o Operador Ternário

```
``Texto ${variavel}``
```

Crases:
Enquadram a string.

Interpolação: Injeta JavaScript diretamente dentro de uma string de texto sem usar sinal de adição.

```
condição ? 'verdadeiro' : 'falso'
```



Ferramentas avançadas que reduzem múltiplas linhas de código (if/else tradicionais) em lógicas precisas de uma única linha.

NotebookLM

Contabilizando Tentativas Dinamicamente

Step 1: Incremento

```
tentativas++
```

→ Ocorre a cada erro.

Step 2: Decisão Gramatical

```
let palavra = tentativas > 1 ? 'tentativas' : 'tentativa';
```

↳ Determina a forma plural ou singular com base na contagem.

Step 3: Interpolação

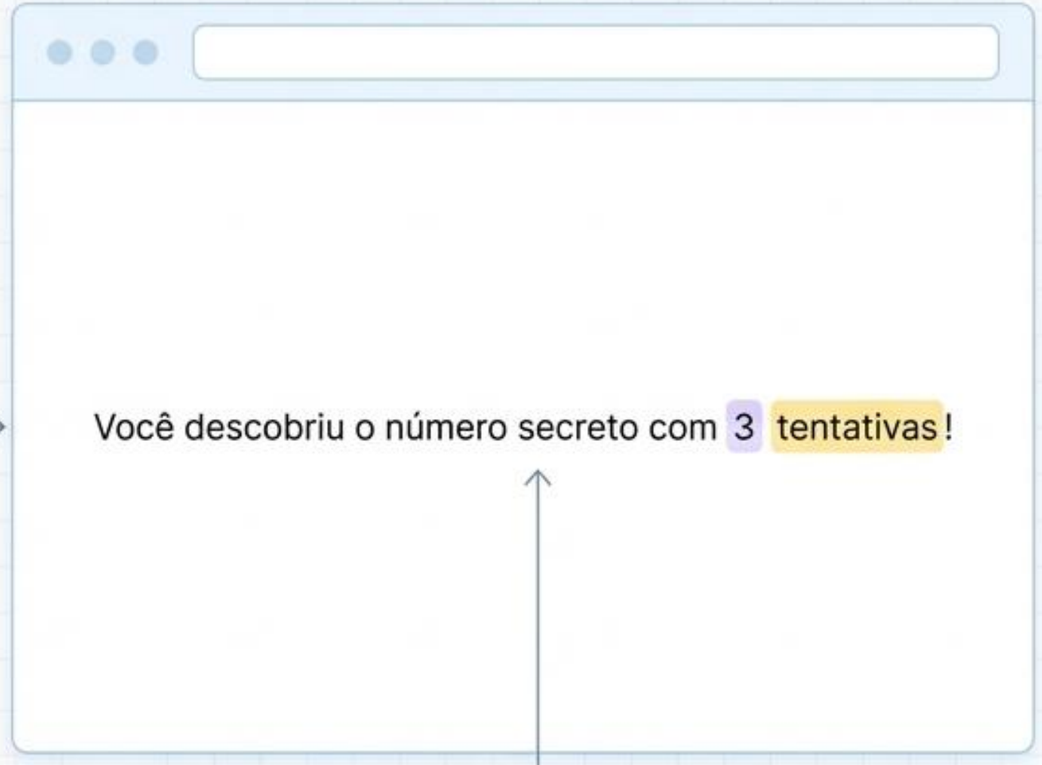
```
let mensagem = `Você descobriu o número secreto com
${tentativas} ${palavra}!`;
```

↳ Gera uma string dinâmica combinando texto e variáveis.

Step 4: Exibição

```
exibirTextoNaTela('p', mensagem);
```

↳ Invoca a função para mostrar o resultado no elemento <p>.



NotebookLM

Buscando por ID e Manipulando Atributos HTML

Targeting Rule

Seletor Geral	Seletor Específico por ID
`querySelector('.classe / tag')`	`getElementById('nome_do_id')`
Seleciona o primeiro elemento que corresponde a um seletor CSS.	getElementById localiza um identificador único no HTML, ignorando a sintaxe de seletores CSS.

State Manipulation Visualizer

```
document.getElementById('id_unico').removeAttribute('disabled');
```

Antes:

removeAttribute('disabled') remove o atributo, desbloqueando o elemento.

Depois:

NotebookLM

Reiniciando o Ciclo do Jogo

Ação 1: Limpando o Input

```
let chute = document.querySelector('input');
chute.value = '';
```

Ação 2: Habilitando o Botão de Reinício

```
document.getElementById('reiniciar').
removeAttribute('disabled');
```

Ao manipular propriedades dinâmicas (.value) e atributos rígidos (disabled), controlamos com precisão o fluxo de vida e o estado da aplicação web.

NotebookLM